

SERVICEHUB — MASTER ROADMAP & PREPARATION PLAN

A local services marketplace and subscription platform for Milk, Tiffin/Mess, Water, and Home Cleaning services — India-focused (₹ INR, English/Hindi/Marathi).

A. Executive Summary

ServiceHub is a three-sided marketplace (Customer / Provider / Admin) built on **React + Vite (frontend)**, **Node.js + Express + Sequelize (backend)**, and **MySQL 8+ (database)**, with **Razorpay** for payments. The platform's core differentiator versus a generic marketplace is a **recurring subscription engine** (daily/weekly/monthly delivery scheduling with pause/resume/skip/vacation logic) layered on top of standard e-commerce/booking flows, combined with a **provider accounting system** that must never conflate "earnings" with "profit."

This document is the single source of truth for implementation. It fixes:

The database schema (30+ tables)

The API surface (versioned, role-gated)

The subscription/delivery scheduler algorithm

The payment and settlement lifecycle

The provider accounting formulas

The i18n, security, testing, and deployment strategy

A 28-phase build plan with acceptance criteria per phase

Any implementer (human or AI coding agent) should be able to execute this document phase-by-phase without inventing architecture.

B. Complete Functional Requirements

B.1 Core Domains

Milk — recurring subscription-first, provider-defined SKUs (Cow/Buffalo/Toned/Organic/Other), morning/evening delivery, quantity in liters.

Tiffin/Mess — meal-plan subscriptions (Breakfast/Lunch/Dinner, Veg/Non-Veg), delivery-day selection, renewal.

Water — recurring subscription **and** one-time/emergency on-demand orders (10L/20L/RO/Mineral cans).

Cleaning — one-time bookings **and** recurring cleaning plans, slot-based (uses Provider Availability), property-attribute intake (rooms/bathrooms/kitchens/floor).

B.2 Cross-Cutting Requirements

Subscription lifecycle: create, pause, resume, skip (single date), vacation mode (date range), cancel, expire, renew.

Delivery lifecycle: Scheduled → Out for Delivery → Delivered / Skipped / Cancelled.

Order lifecycle (one-time, incl. cleaning & emergency water): Pending → Confirmed → In Progress → Completed / Cancelled.

Provider service-area validation (pincode-based) enforced **before** any subscription/order is allowed.

Provider availability (weekly slot grid) enforced for **Cleaning** bookings.

Wallet (customer) for credits/refunds; NOT a second source of truth for provider financials.

Coupons with abuse prevention (usage limits, per-user limits).

Reviews restricted to eligible completed orders/deliveries.

In-app notifications, extensible to email/SMS/push.

Support tickets: Customer↔Admin and Provider↔Admin threads.

Full audit logging of admin/financial actions.

i18n: en/hi/mr, 100% of user-facing strings, persisted language preference, English fallback.

B.3 Explicit Non-Requirements (v1)

No separate Delivery Partner role (Section 48 — provider self-delivers).

No mandatory GPS/Google Maps (pincode-based area matching only, schema GPS-ready).

No multi-currency (INR only).

C. User Roles and Permissions

Capability	Customer	Provider	Admin
Register/Login	✓ self	✓ self (then KYC-gated)	✗ seeded/created by super-admin
Browse/search services	✓	— (own only)	✓ (all)
Subscribe/Book	✓	✗	✗
Manage own services/pricing	✗	✓ (after approval)	✓ (moderation)
Mark delivery status	✗	✓ (own orders)	✓ (override)
View own financials	✗	✓ (own)	✓ (all, platform-wide)
Approve KYC / providers	✗	✗	✓
Configure commission	✗	✗	✓
Process settlements	✗	request only	✓ approve/reject
Manage coupons	use only	✗	✓ create/manage
Moderate reviews	own review only	reply only	✓ full moderation
Audit logs	✗	✗	✓

RBAC is enforced at the middleware layer (see Section N) using a `roles` table + `users.role_id`, never inferred from the frontend.

Role precedence for route guards: `authenticate` (JWT) → `authorize(['admin'])` / `authorize(['provider'])` / `authorize(['customer'])` → `ownershipCheck` (e.g., a provider can only mutate their own `service.provider_id`).

D. Business Model

Commission-based marketplace: Admin configures commission % (global → category → service-specific override, see Section T).

Provider Earnings = Gross Sales – Platform Commission – Refunds (money the provider is entitled to from platform transactions).

Provider Net Profit = Provider Earnings – Provider Expenses (real business profit, includes provider's own operating costs like fuel, ingredients, staff).

Settlement: Admin periodically pays out `Available Settlement` (Earnings not yet paid, net of any settlement-adjust items) to the provider's bank account (masked, encrypted at rest).

Coupons are funded by the platform or provider depending on `funded_by` (flagged per coupon) — this is called out explicitly in Section 47's gap analysis below (see Section Z gap notes).

E. Complete User Journeys

E.1 Customer — Subscription (Milk/Tiffin/Water)

Register → Verify → Add Address → Browse Category → Filter by Pincode-served Providers → View Provider Profile (ratings, KYC badge) → Select Plan (qty, time slot, frequency) → Choose Start Date → Apply Coupon → Razorpay Checkout → Backend Verification → Subscription Activated → Scheduler generates SubscriptionDeliveries → Daily delivery tracking → Pause/Skip/Vacation as needed → Renewal billing at period end → Review after N completed deliveries.

E.2 Customer — Cleaning (One-time)

Register → Add Address → Browse Cleaning → Select Provider (by pincode + availability) → Choose Service Type (Regular/Deep/Bathroom/Kitchen/Sofa) → Enter Property Details (rooms, bathrooms, kitchens, floor, instructions) → Pick Slot from Provider Availability → Coupon → Payment → Order Confirmed → Provider marks In Progress → Completed → Review.

E.3 Provider Onboarding

Register → Business Profile → KYC Upload (ID, address proof, bank details) → Pending Admin Review → Approved/Rejected (with reason, audit-logged) → Add Services (per category) → Set Pricing & Subscription Plans → Define Service Areas (pincodes) → Define Availability (for Cleaning) → Go Live.

E.4 Provider Operations

Receive Order/Subscription Delivery Task → Prepare → Out for Delivery → Delivered (or Skipped/Cancelled with reason) → Revenue auto-recorded on payment capture → Add Expenses (fuel/ingredients/staff/etc.) → View Today/Period Accounting (Sales, Commission, Refunds, Earnings, Expenses, Net Profit) → Request Settlement → Admin Approves → Payout recorded.

E.5 Admin Operations

Login → Dashboard (platform KPIs) → Review Pending KYC → Approve/Reject → Configure Commission Hierarchy → Monitor Orders/Subscriptions/Deliveries in real time → Handle Refund Requests → Process Settlement Requests → Manage Coupons/Categories/Service Areas → Moderate Reviews → Resolve Support Tickets → Export Reports → Review Audit Logs.

F. System Architecture



React (Vite) SPA |
Customer / Provider / |
Admin bundles |
react-router + i18next |

| Axios (JWT bearer)



Express.js REST API |
/api/v1/* |
middlewares: auth, RBAC, |
validation, rate-limit, |
helmet, CORS |

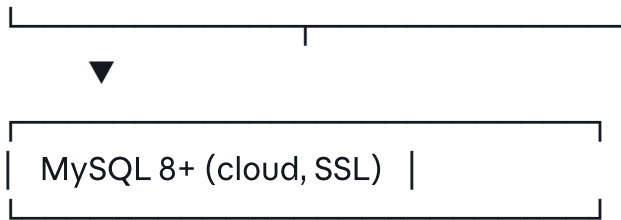
|



Services (business logic)	node-cron Scheduler Job (subscription delivery gen)	Razorpay SDK (orders, verify, refunds)
---------------------------------	--	---



Sequelize ORM |
(models, migrations, |
transactions) |



Design principles:

Single source of truth: MySQL. No client-side caching of financial state beyond display.

Stateless API: JWT access tokens (short-lived) + refresh tokens (rotated, stored hashed).

Idempotent scheduler: `UNIQUE(subscription_id, delivery_date)` guarantees no duplicate delivery rows even on re-run.

Money never trusted from client: All order/subscription totals are recomputed server-side from `services`/`pricing` tables before Razorpay order creation.

Layered backend: `routes → controllers → services → models`. Controllers only orchestrate; business/financial logic lives in `services/`.

G. Technology Stack (Fixed — Non-Negotiable)

Layer	Technology
Frontend	React 18, Vite, React Router v6, Axios, react-i18next
Backend	Node.js (LTS), Express.js
ORM	Sequelize (+ sequelize-cli for migrations/seeder)
Database	MySQL 8+ (cloud-hosted, SSL required) — no SQLite/Mongo/Firebase/Postgres
Auth	JWT (access + refresh), bcrypt
Payments	Razorpay (Orders API + Webhooks)
Uploads	Multer (local disk v1, interface abstracted for future S3/Cloud Storage)
Scheduler	node-cron
PDF (invoices)	pdfkit or puppeteer (headless) — pdfkit recommended for lighter footprint
Validation	express-validator or Joi (pick one, apply consistently — recommend Joi for schema reuse)
Security middleware	helmet, cors, express-rate-limit
Testing	Jest + Supertest (backend), Vitest + React Testing Library (frontend), Playwright (E2E)

H. Complete Database Architecture (Overview)

35 tables grouped by domain. All monetary columns use `DECIMAL(12,2)`. All tables have `id` (BIGINT UNSIGNED AUTO_INCREMENT PK), `created_at`, `updated_at` (and `deleted_at` where soft-delete is specified) unless noted.

AUTH: roles, users **CUSTOMER:** customers, addresses, wallets, wallet_transactions **PROVIDER:** providers, provider_bank_accounts, provider_availability, provider_expenses, provider_earnings, provider_settlements **CATALOG:** categories, services, service_plans, service_areas

SUBSCRIPTIONS: customer_subscriptions, subscription_deliveries, skipped_deliveries, subscription_payments **ORDERS:** orders, order_items
PAYMENTS: payments, refunds **BILLING:** invoices, coupons, coupon_usage **COMMUNICATION:** reviews, notifications, support_tickets,
support_messages **SECURITY:** audit_logs **PLATFORM CONFIG (gap-filled — see Section Z notes):** commission_rules, platform_settings,
kyc_documents

Total: **31 tables**. Rationale for the 4 tables added beyond the prompt's list is given inline in each table's Purpose and summarized in the Gap Analysis (Section Z).

I. Complete Table-by-Table Schema

I.1 AUTH

roles

Purpose: Fixed set of system roles for RBAC.

Columns: id PK, name VARCHAR(30) UNIQUE NOT NULL (customer, provider, admin), description VARCHAR(255) NULL.

Seed only, no runtime CRUD from UI (except maybe super-admin, out of scope v1).

users

Purpose: Single auth identity table for all roles (email/phone + password).

Columns:

id PK

role_id FK→roles.id NOT NULL

full_name VARCHAR(120) NOT NULL

email VARCHAR(150) UNIQUE NOT NULL, INDEX

phone VARCHAR(15) UNIQUE NOT NULL, INDEX

password_hash VARCHAR(255) NOT NULL (bcrypt)

preferred_language ENUM('en','hi','mr') DEFAULT 'en'

status ENUM('active','suspended','deleted') DEFAULT 'active'

email_verified_at DATETIME NULL

phone_verified_at DATETIME NULL

refresh_token_hash VARCHAR(255) NULL

last_login_at DATETIME NULL

created_at, updated_at

Indexes: UNIQUE(email), UNIQUE(phone), INDEX(role_id, status)

I.2 CUSTOMER

customers (1:1 extension of users where role=customer)

id PK, **user_id** FK→users.id UNIQUE NOT NULL

default_address_id FK→addresses.id NULL

date_of_birth DATE NULL

referral_code VARCHAR(20) UNIQUE NULL

created_at, **updated_at**

addresses

id PK, **customer_id** FK→customers.id NOT NULL, INDEX

label VARCHAR(30) NOT NULL (Home/Work/Other)

house_no VARCHAR(50), **building** VARCHAR(100), **street** VARCHAR(150)

area VARCHAR(100) NOT NULL, INDEX

city VARCHAR(100) NOT NULL, INDEX

state VARCHAR(100) NOT NULL

pincode VARCHAR(10) NOT NULL, INDEX

latitude DECIMAL(10,7) NULL, **longitude** DECIMAL(10,7) NULL (*GPS-ready, unused v1*)

is_default BOOLEAN DEFAULT FALSE

Indexes: INDEX(customer_id), INDEX(pincode)

wallets

id PK, **customer_id** FK→customers.id UNIQUE NOT NULL

balance DECIMAL(12,2) NOT NULL DEFAULT 0.00

updated_at

Balance is a **derived cache**; **wallet_transactions** is the ledger of record. Every balance change happens inside a DB transaction that also inserts a

wallet_transactions row (never update **balance** standalone).

wallet_transactions

id PK, **wallet_id** FK→wallets.id NOT NULL, INDEX

type ENUM('credit','debit','refund') NOT NULL

amount DECIMAL(12,2) NOT NULL

balance_after DECIMAL(12,2) NOT NULL

reference_type ENUM('order','subscription_payment','refund','manual_admin_adjustment') NOT NULL

`reference_id` BIGINT UNSIGNED NULL
`description` VARCHAR(255)
`created_at`

Indexes: INDEX(wallet_id, created_at)

I.3 PROVIDER

`providers`

`id` PK, `user_id` FK→users.id UNIQUE NOT NULL

`business_name` VARCHAR(150) NOT NULL

`business_description` TEXT NULL

`logo_url` VARCHAR(255) NULL

`kyc_status` ENUM('pending','approved','rejected') DEFAULT 'pending', INDEX

`kyc_rejection_reason` VARCHAR(255) NULL

`is_active` BOOLEAN DEFAULT TRUE (provider can deactivate storefront)

`average_rating` DECIMAL(3,2) DEFAULT 0.00 (*denormalized, recomputed on review insert*)

`created_at`, `updated_at`

`kyc_documents` (*gap-fill: prompt says "Upload KYC" but specifies no table — needed for auditable multi-document KYC*)

`id` PK, `provider_id` FK→providers.id NOT NULL, INDEX

`document_type` ENUM('id_proof','address_proof','bank_proof','business_license','other') NOT NULL

`file_url` VARCHAR(255) NOT NULL

`status` ENUM('pending','approved','rejected') DEFAULT 'pending'

`reviewed_by` FK→users.id NULL (admin), `reviewed_at` DATETIME NULL

`created_at`

`provider_bank_accounts`

`id` PK, `provider_id` FK→providers.id UNIQUE NOT NULL

`account_holder_name` VARCHAR(120) NOT NULL

`account_number_encrypted` VARBINARY(255) NOT NULL (*AES-256 encrypted at application layer; never store plaintext*)

`account_number_last4` CHAR(4) NOT NULL (*for masked UI display, e.g. "●●●● 4321"*)

`ifsc_code` VARCHAR(11) NOT NULL

`bank_name` VARCHAR(100) NOT NULL

`verified` BOOLEAN DEFAULT FALSE

`created_at`, `updated_at`

`provider_availability`

`id` PK, `provider_id` FK→providers.id NOT NULL, INDEX

`day_of_week` TINYINT NOT NULL (0=Sun..6=Sat)

`start_time` TIME NOT NULL

`end_time` TIME NOT NULL

`slot_duration_minutes` SMALLINT NOT NULL DEFAULT 60

`max_bookings_per_slot` SMALLINT NOT NULL DEFAULT 1

`is_available` BOOLEAN DEFAULT TRUE

Indexes: INDEX(provider_id, day_of_week)

Used primarily by Cleaning bookings; other categories may ignore.

`provider_expenses`

`id` PK, `provider_id` FK→providers.id NOT NULL, INDEX

`category` ENUM('fuel','ingredients','raw_materials','cleaning_supplies','staff_salary','maintenance','packaging','transportation','other') NOT NULL

`amount` DECIMAL(12,2) NOT NULL CHECK (amount > 0)

`expense_date` DATE NOT NULL, INDEX

`description` VARCHAR(255) NULL

`receipt_url` VARCHAR(255) NULL

`created_at`, `updated_at`

Indexes: INDEX(provider_id, expense_date)

`provider_earnings`

Purpose: **Immutable ledger row per completed, revenue-generating event** (one row per order/subscription-delivery payment capture). This is what accounting reports SUM over — never recompute earnings by re-scanning `payments` + `orders` joins at query time for historical periods where commission may have since changed (see Section T).

`id` PK, `provider_id` FK→providers.id NOT NULL, INDEX

`source_type` ENUM('order','subscription_delivery') NOT NULL

`source_id` BIGINT UNSIGNED NOT NULL

`payment_id` FK→payments.id NOT NULL

`gross_amount` DECIMAL(12,2) NOT NULL

`commission_rate_applied` DECIMAL(5,2) NOT NULL (*snapshot at transaction time*)

`commission_amount` DECIMAL(12,2) NOT NULL

`refund_amount` DECIMAL(12,2) NOT NULL DEFAULT 0.00 (*updated if a later refund applies against this earning row*)
`net_earning` DECIMAL(12,2) NOT NULL (*gross – commission – refund_amount*)
`earning_date` DATE NOT NULL, INDEX
`settlement_id` FK→provider_settlements.id NULL (*NULL = unsettled, part of Pending/Available Settlement*)
`created_at`

Indexes: INDEX(provider_id, earning_date), INDEX(settlement_id)

`provider_settlements`

`id` PK, `provider_id` FK→providers.id NOT NULL, INDEX
`period_start` DATE NOT NULL, `period_end` DATE NOT NULL
`total_earnings` DECIMAL(12,2) NOT NULL (*SUM of net_earning for included provider_earnings rows*)
`status` ENUM('requested','approved','paid','rejected') DEFAULT 'requested', INDEX
`requested_at` DATETIME NOT NULL
`processed_by` FK→users.id NULL (admin)
`processed_at` DATETIME NULL
`payout_reference` VARCHAR(100) NULL (*bank transfer UTR etc.*)
`rejection_reason` VARCHAR(255) NULL
`created_at`, `updated_at`

I.4 CATALOG

`categories`

`id` PK, `name` VARCHAR(50) UNIQUE NOT NULL (Milk/Tiffin/Water/Cleaning), `slug` VARCHAR(50) UNIQUE, `icon_url` VARCHAR(255), `is_active` BOOLEAN DEFAULT TRUE

`services`

`id` PK, `provider_id` FK→providers.id NOT NULL, INDEX
`category_id` FK→categories.id NOT NULL, INDEX
`name` VARCHAR(150) NOT NULL
`description` TEXT NULL
`type` ENUM('subscription','one_time','both') NOT NULL
`base_price` DECIMAL(10,2) NOT NULL
`unit` VARCHAR(30) NOT NULL (liter/plate/can/service)
`attributes_json` JSON NULL (*category-specific: milk_type, meal_type veg/nonveg, water_size, cleaning_type*)

`is_active` BOOLEAN DEFAULT TRUE, INDEX

`image_url` VARCHAR(255) NULL

Indexes: INDEX(provider_id, category_id, is_active)

`service_plans` (gap-fill: prompt requires "Configure subscription plans" per-service but has no dedicated table — folding frequency/pricing into

`services` alone would prevent one service from offering multiple frequency tiers, e.g., Daily ₹40/L vs Monthly-prepaid ₹38/L)

`id` PK, `service_id` FK→services.id NOT NULL, INDEX

`frequency` ENUM('daily','weekly','monthly','custom') NOT NULL

`price` DECIMAL(10,2) NOT NULL

`min_quantity` DECIMAL(6,2) DEFAULT 1

`billing_cycle_days` SMALLINT NOT NULL (1/7/30/custom)

`is_active` BOOLEAN DEFAULT TRUE

`service_areas`

`id` PK, `provider_id` FK→providers.id NOT NULL, INDEX

`state` VARCHAR(100) NOT NULL, `city` VARCHAR(100) NOT NULL, `area` VARCHAR(100) NULL

`pincode` VARCHAR(10) NOT NULL, INDEX

Indexes: UNIQUE(provider_id, pincode)

I.5 SUBSCRIPTIONS

`customer_subscriptions`

`id` PK, `customer_id` FK→customers.id NOT NULL, INDEX

`provider_id` FK→providers.id NOT NULL, INDEX

`service_id` FK→services.id NOT NULL

`service_plan_id` FK→service_plans.id NOT NULL

`address_id` FK→addresses.id NOT NULL

`quantity` DECIMAL(6,2) NOT NULL

`delivery_time_slot` ENUM('morning','evening','custom') NOT NULL

`custom_time` TIME NULL

`start_date` DATE NOT NULL

`end_date` DATE NULL (NULL = open-ended until cancelled)

`status` ENUM('active','paused','vacation','cancelled','expired') DEFAULT 'active', INDEX

`vacation_start` DATE NULL, `vacation_end` DATE NULL

`next_billing_date` DATE NOT NULL, INDEX

`created_at`, `updated_at`

Indexes: INDEX(status, next_billing_date), INDEX(customer_id, status)

subscription_deliveries

`id` PK, `subscription_id` FK→customer_subscriptions.id NOT NULL, INDEX

`delivery_date` DATE NOT NULL

`status` ENUM('scheduled','out_for_delivery','delivered','skipped','cancelled') DEFAULT 'scheduled', INDEX

`quantity` DECIMAL(6,2) NOT NULL (*snapshot, in case subscription qty later changes*)

`delivered_at` DATETIME NULL

`notes` VARCHAR(255) NULL

Indexes: **UNIQUE(subscription_id, delivery_date)** (mandatory per spec, prevents duplicate generation), INDEX(delivery_date, status)

skipped_deliveries

`id` PK, `subscription_id` FK→customer_subscriptions.id NOT NULL, INDEX

`skip_date` DATE NOT NULL

`reason` VARCHAR(255) NULL

`created_at`

Indexes: UNIQUE(subscription_id, skip_date) — checked by scheduler before generating a delivery row.

subscription_payments

`id` PK, `subscription_id` FK→customer_subscriptions.id NOT NULL, INDEX

`billing_period_start` DATE NOT NULL, `billing_period_end` DATE NOT NULL

`amount` DECIMAL(10,2) NOT NULL

`payment_id` FK→payments.id NULL (*NULL until paid*)

`status` ENUM('pending','paid','failed','retrying') DEFAULT 'pending', INDEX

`due_date` DATE NOT NULL

`retry_count` TINYINT DEFAULT 0

`created_at`, `updated_at`

I.6 ORDERS

orders (*one-time purchases: Cleaning bookings, emergency Water orders, any non-subscription checkout*)

`id` PK, `order_number` VARCHAR(20) UNIQUE NOT NULL (*human-readable, e.g. SH-2026-000123*)

`customer_id` FK→customers.id NOT NULL, INDEX

`provider_id` FK→providers.id NOT NULL, INDEX
`address_id` FK→addresses.id NOT NULL
`category_id` FK→categories.id NOT NULL
`status` ENUM('pending','confirmed','in_progress','completed','cancelled') DEFAULT 'pending', INDEX
`subtotal` DECIMAL(12,2) NOT NULL
`discount_amount` DECIMAL(12,2) DEFAULT 0.00
`total_amount` DECIMAL(12,2) NOT NULL
`coupon_id` FK→coupons.id NULL
`scheduled_date` DATE NOT NULL, `scheduled_time_slot` VARCHAR(30) NULL
`booking_details_json` JSON NULL (*cleaning: rooms/bathrooms/kitchens/floor/instructions/duration*)
`payment_id` FK→payments.id NULL
`cancelled_reason` VARCHAR(255) NULL
`created_at`, `updated_at`

Indexes: INDEX(customer_id, status), INDEX(provider_id, status), INDEX(scheduled_date)

`order_items`

`id` PK, `order_id` FK→orders.id NOT NULL, INDEX
`service_id` FK→services.id NOT NULL
`quantity` DECIMAL(6,2) NOT NULL
`unit_price` DECIMAL(10,2) NOT NULL
`line_total` DECIMAL(12,2) NOT NULL

I.7 PAYMENTS

`payments`

`id` PK, `customer_id` FK→customers.id NOT NULL, INDEX
`reference_type` ENUM('order','subscription_payment') NOT NULL
`reference_id` BIGINT UNSIGNED NOT NULL
`amount` DECIMAL(12,2) NOT NULL
`currency` CHAR(3) DEFAULT 'INR'
`razorpay_order_id` VARCHAR(50) UNIQUE NOT NULL, INDEX
`razorpay_payment_id` VARCHAR(50) UNIQUE NULL
`razorpay_signature` VARCHAR(255) NULL

status ENUM('created','pending','paid','failed','refunded','partially_refunded') DEFAULT 'created', INDEX
method VARCHAR(30) NULL (card/upi/netbanking/wallet)
idempotency_key VARCHAR(100) UNIQUE NOT NULL (*prevents duplicate order creation on client retry*)
paid_at DATETIME NULL
created_at, **updated_at**

refunds

id PK, **payment_id** FK→payments.id NOT NULL, INDEX
amount DECIMAL(12,2) NOT NULL
reason VARCHAR(255) NOT NULL
razorpay_refund_id VARCHAR(50) UNIQUE NULL
status ENUM('initiated','processed','failed') DEFAULT 'initiated', INDEX
initiated_by FK→users.id NOT NULL (admin)
processed_at DATETIME NULL
created_at

I.8 BILLING

invoices

id PK, **invoice_number** VARCHAR(20) UNIQUE NOT NULL
customer_id FK→customers.id NOT NULL, INDEX
provider_id FK→providers.id NOT NULL
reference_type ENUM('order','subscription_payment') NOT NULL
reference_id BIGINT UNSIGNED NOT NULL
subtotal DECIMAL(12,2), **discount** DECIMAL(12,2) DEFAULT 0, **tax** DECIMAL(12,2) DEFAULT 0, **total** DECIMAL(12,2) NOT NULL
payment_status ENUM('paid','unpaid','refunded') NOT NULL
pdf_url VARCHAR(255) NULL
issued_at DATETIME NOT NULL

coupons

id PK, **code** VARCHAR(30) UNIQUE NOT NULL, INDEX
discount_type ENUM('percentage','fixed') NOT NULL
discount_value DECIMAL(10,2) NOT NULL
min_order_amount DECIMAL(10,2) DEFAULT 0

`max_discount_amount` DECIMAL(10,2) NULL (*caps percentage coupons*)

`start_date` DATE NOT NULL, `end_date` DATE NOT NULL

`usage_limit_total` INT NULL, `usage_limit_per_user` INT DEFAULT 1

`category_id` FK→categories.id NULL (*NULL = all categories*)

`is_active` BOOLEAN DEFAULT TRUE

`created_by` FK→users.id NOT NULL

`coupon_usage`

`id` PK, `coupon_id` FK→coupons.id NOT NULL, `customer_id` FK→customers.id NOT NULL

`order_id` FK→orders.id NULL, `subscription_payment_id` FK→subscription_payments.id NULL

`discount_applied` DECIMAL(10,2) NOT NULL, `used_at` DATETIME NOT NULL

Indexes: INDEX(coupon_id, customer_id) — used to enforce per-user limit.

I.9 COMMUNICATION

`reviews`

`id` PK, `customer_id` FK→customers.id NOT NULL

`provider_id` FK→providers.id NOT NULL, INDEX

`reference_type` ENUM('order','subscription') NOT NULL, `reference_id` BIGINT UNSIGNED NOT NULL

`rating` TINYINT NOT NULL CHECK (rating BETWEEN 1 AND 5)

`comment` TEXT NULL

`provider_reply` TEXT NULL, `provider_replied_at` DATETIME NULL

`moderation_status` ENUM('visible','hidden','reported') DEFAULT 'visible'

`created_at`

Indexes: UNIQUE(customer_id, reference_type, reference_id) — one review per completed thing.

`notifications`

`id` PK, `user_id` FK→users.id NOT NULL, INDEX

`type` VARCHAR(50) NOT NULL (order_confirmed/subscription_paused/payment_failed/kyc_approved/... — full enum in Section U)

`title` VARCHAR(150), `body` VARCHAR(500)

`reference_type` VARCHAR(30) NULL, `reference_id` BIGINT UNSIGNED NULL

`is_read` BOOLEAN DEFAULT FALSE, INDEX

`created_at`

Indexes: INDEX(user_id, is_read, created_at)

support_tickets

`id` PK, `ticket_code` VARCHAR(20) UNIQUE NOT NULL
`user_id` FK→users.id NOT NULL, INDEX
`subject` VARCHAR(150) NOT NULL
`category` ENUM('order','subscription','payment','kyc','other') NOT NULL
`priority` ENUM('low','medium','high') DEFAULT 'medium'
`status` ENUM('open','in_progress','resolved','closed') DEFAULT 'open', INDEX
`created_at`, `updated_at`

support_messages

`id` PK, `ticket_id` FK→support_tickets.id NOT NULL, INDEX
`sender_id` FK→users.id NOT NULL
`message` TEXT NOT NULL
`attachment_url` VARCHAR(255) NULL
`created_at`

I.10 SECURITY & CONFIG

audit_logs

`id` PK, `user_id` FK→users.id NULL (system actions may be NULL)
`action` VARCHAR(100) NOT NULL (e.g. `provider.approved`, `commission.updated`, `settlement.approved`)
`entity_type` VARCHAR(50) NOT NULL, `entity_id` BIGINT UNSIGNED NOT NULL
`old_values_json` JSON NULL, `new_values_json` JSON NULL
`ip_address` VARCHAR(45) NULL
`created_at`

Indexes: INDEX(entity_type, entity_id), INDEX(user_id, created_at)

commission_rules (gap-fill — required by Section 47's hierarchy but no table was named)

`id` PK, `scope` ENUM('global','category','service') NOT NULL
`category_id` FK→categories.id NULL, `service_id` FK→services.id NULL
`commission_percent` DECIMAL(5,2) NOT NULL
`effective_from` DATETIME NOT NULL, `effective_to` DATETIME NULL (NULL = currently active)
`created_by` FK→users.id NOT NULL

Indexes: INDEX(scope, category_id, service_id, effective_from)

Resolution order at transaction time (see Section T): service-scoped active rule → category-scoped active rule → global active rule.

`platform_settings` (gap-fill — needed for "Manage platform settings" Admin capability, key-value config)

`id` PK, `key` VARCHAR(100) UNIQUE NOT NULL, `value` TEXT NOT NULL, `updated_by` FK→users.id, `updated_at`

J. Entity Relationship Explanation

`users` is the auth root; `customers`/`providers` are 1:1 role-profile extensions (chosen over storing role-specific columns directly on `users` to keep the auth table lean and role-agnostic).

`services` belongs to exactly one `provider` and one `category`; `service_plans` gives each service 1..N frequency/price tiers (so "Cow Milk" can have Daily and Monthly pricing simultaneously).

`customer_subscriptions` is the anchor of the recurring engine: it references a specific `service_plan_id` (locking in the plan/price at subscribe time — price changes on the service don't silently alter an active subscriber's rate) and generates many `subscription_deliveries` (1 per delivery date) and many `subscription_payments` (1 per billing cycle).

`orders`/`order_items` handle everything non-recurring: Cleaning bookings and one-time Water orders. `order_items` exists even though most orders have one line item, to keep the schema uniform and future-proof for multi-item carts.

`payments` is polymorphic via `reference_type`/`reference_id`, referenced by both `orders` and `subscription_payments` — this avoids two near-duplicate payment tables.

`provider_earnings` is **derived but persisted** at the moment a payment is captured (not computed live from `payments` joins), specifically so that a later change to `commission_rules` does not retroactively change historical provider earnings (Section 47 requirement). `commission_rate_applied` is a snapshot column for this reason.

`provider_settlements` batches unsettled `provider_earnings` rows (`settlement_id IS NULL`) into a payout; once included, `provider_earnings.settlement_id` is set, removing them from "Available Settlement."

`refunds` references `payments`, not `orders` directly, because a subscription_payment can also be refunded — and a refund must decrement the corresponding `provider_earnings.refund_amount` row via the refund-processing service.

`wallet_transactions` is the ledger; `wallets.balance` is a cached, always-recomputable projection — never treat `wallets.balance` as authoritative if it and the transaction sum disagree (log to `audit_logs` and reconcile).

K. Backend Architecture



```
backend/
├── src/
│   ├── config/      # db.js (Sequelize+SSL), razorpay.js, env loader, i18n(server-side error msgs)
│   ├── models/      # one file per table, associations in models/index.js
│   ├── controllers/  # thin: parse req → call service → shape res
│   ├── services/     # ALL business/financial logic (subscriptionService, accountingService, paymentService...)
│   ├── routes/       # one router per resource, mounted under /api/v1
│   ├── middlewares/   # auth.js, rbac.js, validate.js, rateLimit.js, errorHandler.js, upload.js
│   ├── validators/    # Joi schemas per resource
│   ├── jobs/         # scheduler.js (node-cron), subscriptionDeliveryJob.js, billingRetryJob.js
│   ├── utils/        # money.js (decimal-safe helpers), idempotency.js, encryption.js, pagination.js
│   └── app.js
├── migrations/      # sequelize-cli, timestamped, one change-set per file
├── seeders/         # realistic Indian seed data (Section 51)
├── uploads/         # multer disk storage (KYC docs, receipts, review images)
├── tests/           # unit/ + integration/
├── .env.example
└── package.json
```

Directory responsibilities:

`services/` is where money math lives, deliberately separated from `controllers/` so it's unit-testable without HTTP mocking (Section 36 requires unit tests on "Accounting formulas" and "Subscription calculations" — this structure makes that trivial).

`jobs/` is isolated from `routes/` but the manual trigger endpoint `POST /api/v1/subscriptions/run-scheduler` calls the same `jobs/subscriptionDeliveryJob.js` function the cron calls — one code path, two triggers.

`utils/money.js` wraps all arithmetic to operate on integer paise internally (avoiding floating point) then formats to `DECIMAL(12,2)/₹` at the boundary.

L. Frontend Architecture



```
frontend/
├── src/
│   ├── assets/      # images, icons per category (milk/tiffin/water/cleaning)
│   ├── components/  # shared: Button, Card, Table, StatusBadge, EmptyState, Skeleton, Modal
│   ├── layouts/     # CustomerLayout, ProviderLayout (dashboard shell), AdminLayout (console shell)
│   ├── pages/       # route-level, grouped customer/, provider/, admin/
│   ├── services/    # api/ axios instances + per-resource API clients (authApi, subscriptionApi...)
│   ├── hooks/       # useAuth, useSubscription, usePagination, useDebounce
│   ├── context/     # AuthContext, LanguageContext, ToastContext
│   ├── locales/     # en.json, hi.json, mr.json (namespaced: common, customer, provider, admin)
│   ├── routes/      # AppRoutes.jsx with role-guarded <ProtectedRoute>
│   ├── utils/       # formatCurrency (₹, en-IN locale), dateHelpers
│   └── styles/      # design tokens (see frontend-design conventions), globals
├── public/
└── package.json
```

Three visually distinct bundles share the `components/` design system but use different `layouts/`: Customer = marketplace card grids; Provider = dashboard with left nav + KPI cards; Admin = dense data-table console.

M. Complete API Architecture (Representative Set)

All endpoints prefixed `/api/v1/`. Standard envelope (Section 35). Auth =  , role required in brackets.

Authentication

Method	Endpoint	Auth	Notes
POST	<code>/auth/register</code>	—	body: full_name, email, phone, password, role(customer provider)
POST	<code>/auth/login</code>	—	returns access+refresh tokens
POST	<code>/auth/refresh</code>	—	body: refresh_token → new access token; rotates refresh token
POST	<code>/auth/logout</code>		invalidates refresh_token_hash
POST	<code>/auth/forgot-password</code>	—	emails/SMS reset link (rate-limited)
POST	<code>/auth/reset-password</code>	—	token + new password

Customer

Method	Endpoint	Auth	Notes
GET/PUT	<code>/customers/me</code>	 customer	profile
GET/POST	<code>/addresses</code>	 customer	list/create
PUT/DELETE	<code>/addresses/:id</code>	 customer	ownership-checked
GET	<code>/services?category=milk&pincode=411001</code>	—	pincode filters to providers whose service_areas match
GET	<code>/providers/:id</code>	—	public profile incl. rating, KYC badge
POST	<code>/subscriptions</code>	 customer	validates service_area + creates subscription + first billing
GET	<code>/subscriptions</code> / <code>/subscriptions/:id</code>	 customer	
POST	<code>/subscriptions/:id/pause</code> <code>/resume</code> <code>/cancel</code>	 customer	
POST	<code>/subscriptions/:id/skip</code>	 customer	body: date
POST	<code>/subscriptions/:id/vacation</code>	 customer	body: start,end
GET	<code>/subscriptions/:id/calendar</code>	 customer	delivery calendar
POST	<code>/orders</code>	 customer	Cleaning/one-time Water
GET	<code>/orders</code> / <code>/orders/:id</code>	 customer	
POST	<code>/payments/create-order</code>	 customer	creates Razorpay order (server-computed amount)
POST	<code>/payments/verify</code>	 customer	signature verification, idempotent
GET	<code>/wallet</code> / <code>/wallet/transactions</code>	 customer	

Method	Endpoint	Auth	Notes
POST	<code>/coupons/validate</code>	 customer	body: code, cart context → discount preview
GET	<code>/invoices</code> / <code>/invoices/:id/pdf</code>	 customer	
POST	<code>/reviews</code>	 customer	only for eligible completed reference
GET	<code>/notifications</code>	 customer/provider/admin shared endpoint, scoped by JWT user	

Provider				
Method	Endpoint	Auth	Notes	
POST	/providers/kyc	 provider	multipart upload → kyc_documents	
GET/PUT	/providers/me	 provider		
GET/POST	/provider/services	 provider(approved)		
PUT/DELETE	/provider/services/:id	 provider		
POST	/provider/services/:id/plans	 provider	service_plans	
GET/POST	/provider/service-areas	 provider		
GET/POST	/provider/availability	 provider		
GET	/provider/subscriptions /orders /deliveries	 provider	own only	
PUT	/provider/deliveries/:id/status	 provider	scheduled→out_for_delivery→delivered skipped cancelled	
GET/POST	/provider/expenses	 provider		
PUT/DELETE	/provider/expenses/:id	 provider		
GET	/provider/accounting/summary? period=today week month year custom	 provider	Section S formulas	
GET	/provider/analytics/*	 provider	revenue/expense/commission/profit charts	
GET/POST	/provider/settlements	 provider	GET=history, POST=request settlement	

Admin

Method	Endpoint	Auth	Notes
GET	<code>/admin/users</code> <code>/customers</code> <code>/providers</code>	 admin paginated, filterable	
PUT	<code>/admin/providers/:id/kyc</code>	 admin approve/reject, writes audit_log + kyc_documents.status	
GET/POST/PUT	<code>/admin/categories</code>	 admin	
GET/PUT	<code>/admin/services/:id</code>	 admin moderation (deactivate)	
GET/POST	<code>/admin/commission-rules</code>	 admin	
GET	<code>/admin/subscriptions</code> <code>/orders</code> <code>/deliveries</code>	 admin platform-wide monitoring	
POST	<code>/admin/refunds</code>	 admin triggers Razorpay refund + decrements provider_earnings	
GET/PUT	<code>/admin/settlements/:id</code>	 admin approve/reject/mark paid	
GET/POST/PUT	<code>/admin/coupons</code>	 admin	
PUT	<code>/admin/reviews/:id/moderate</code>	 admin	
GET/PUT	<code>/admin/support-tickets</code>	 admin	
GET	<code>/admin/reports/*</code>	 admin revenue, growth, category performance	
GET	<code>/admin/audit-logs</code>	 admin	
GET/PUT	<code>/admin/service-areas</code>	 admin override/inspect	
GET/PUT	<code>/admin/settings</code>	 admin platform_settings	

System

Method	Endpoint	Auth	Notes
POST	<code>/subscriptions/run-scheduler</code>	 admin	manual trigger, same code path as cron
POST	<code>/payments/webhook</code>	signature-verified (Razorpay) webhook-ready endpoint (Section 18)	

Every mutating endpoint: Joi-validated body, RBAC-checked role, ownership-checked (provider can only edit `WHERE provider_id = req.user.provider.id`), and returns the standard envelope with explicit 400/401/403/404/409/422/500 handling.

N. Authentication and RBAC

Access token: JWT, 15 min TTL, payload `{user_id, role, provider_id?|customer_id?}`.

Refresh token: opaque random string, 7-30 day TTL, stored **hashed** (bcrypt) in `users.refresh_token_hash`; rotated on every `/auth/refresh` call (old hash invalidated) to limit replay window.

Password: bcrypt, cost factor 12; reset tokens are single-use, 15-min expiry, hashed at rest.

RBAC middleware chain: `authenticate` (verifies JWT, attaches `req.user`) → `authorize(['provider'])` (checks `req.user.role`) → resource-specific `ownership` middleware (e.g. loads the service and checks `service.provider_id === req.user.provider_id`, else 403).

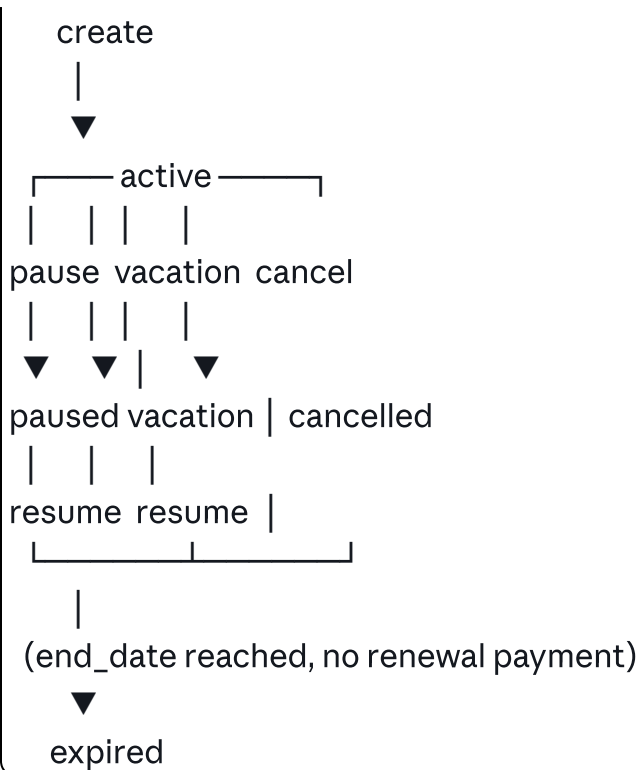
Auth throttling: `express-rate-limit` on `/auth/login` (e.g. 5 attempts / 15 min / IP+email combo) and `/auth/forgot-password`.

Provider gating: even an authenticated provider with `kyc_status='pending'` is blocked (403 `PROVIDER_NOT_APPROVED`) from `/provider/services` write endpoints until `kyc_status='approved'`.

O. Subscription Architecture

State machine for `customer_subscriptions.status`:





`pause/resume`: sets `status='paused'/'active'`; scheduler simply skips paused subscriptions when generating deliveries (no row created — cleanest audit trail is “no delivery exists” rather than a delivery marked skipped for pause, whereas an explicit single-date skip **does** get a `skipped_deliveries` row for the reason/audit trail).

`skip(date)`: inserts into `skipped_deliveries`; scheduler checks this table before generating that date’s delivery.

`vacation(start,end)`: sets `status='vacation'`, `vacation_start/end`; scheduler skips any date within range for that subscription without needing individual skip rows.

`cancel`: terminal; no further deliveries or billing generated; existing scheduled-but-undelivered rows are marked `cancelled`.

`renewal`: when `next_billing_date` arrives, `subscriptionBillingService` creates a new `subscription_payments` row and a Razorpay order; on successful payment, `next_billing_date += billing_cycle_days` and (if using prepaid model) `end_date` extends.

P. Delivery Scheduler Architecture

Trigger: node-cron `0 5 0 * * *` (00:05 daily) calling `jobs/subscriptionDeliveryJob.js::run()`; identical logic reachable via `POST /api/v1/subscriptions/run-scheduler` (admin-only, for manual/ops recovery).

Algorithm (pseudocode):




```
run(targetDate = today):
  subs = CustomerSubscription.findAll({ status: 'active', start_date <= targetDate,
                                         (end_date IS NULL OR end_date >= targetDate) })
  for each sub in subs:
    if targetDate in [sub.vacation_start, sub.vacation_end]: continue
    if SkippedDelivery.exists(sub.id, targetDate): continue
    if not ServiceArea.covers(sub.provider_id, sub.address.pincode):
      flag + notify admin, continue # area coverage changed after subscribe
    if sub.provider requires availability (Cleaning-type) and slot unavailable: continue + notify
  TRANSACTION:
    SubscriptionDelivery.findOrCreate({
      where: { subscription_id: sub.id, delivery_date: targetDate }, # UNIQUE constraint = idempotency guard
      defaults: { status: 'scheduled', quantity: sub.quantity }
    })
  COMMIT
  log summary (created, skipped-vacation, skipped-explicit, area-flagged) to job run log
```

Idempotency: even if the job is triggered twice for the same date (cron misfire + manual admin trigger), `findOrCreate` against the `UNIQUE(subscription_id, delivery_date)` index guarantees exactly one row — the second call finds the existing row and no-ops.

Failure isolation: each subscription is processed in its own try/catch inside the loop; one bad row must not abort the whole run. Failures are collected and surfaced in the job's return summary/log, not swallowed silently.

Provider marks each generated delivery: `scheduled → out_for_delivery → delivered|skipped|cancelled` via `PUT /provider/deliveries/:id/status`; `delivered` is what feeds `provider_earnings` for subscription-based revenue recognition (see Section S).

Q. Razorpay Payment Architecture

Never trust client-reported success (Section 18/46). Flow:

`POST /payments/create-order` — server recomputes the true amount from `service_plans`/`orders` (never accepts `amount` from the request body), creates a Razorpay Order via SDK, persists a `payments` row (`status='created'`) keyed by a server-generated `idempotency_key` derived from `(reference_type, reference_id, customer_id)` so a double-click/retry reuses the same Razorpay order rather than creating a duplicate charge.

Frontend opens Razorpay Checkout with the returned `order_id`.

On completion, frontend calls `POST /payments/verify` with `razorpay_order_id`, `razorpay_payment_id`, `razorpay_signature`.

Backend verifies signature: `HMAC_SHA256(order_id + "|" + payment_id, RAZORPAY_KEY_SECRET) === signature`. If invalid → 400, `payments.status` stays `pending`/`failed`, **no** order/subscription is activated.

If valid, inside a DB transaction: `payments.status='paid'`, `paid_at=now()`; mark `orders.status='confirmed'` or `subscription_payments.status='paid'`; insert `provider_earnings` row using the currently-effective `commission_rules`; insert `invoices` row.

Webhook (`POST /payments/webhook`, Razorpay signature header verified) is the source of truth of last resort — handles cases where the client never calls `/verify` (browser closed mid-flow); it performs the same step-5 logic idempotently (checked via `razorpay_payment_id` uniqueness).

Refunds: Admin-initiated (`POST /admin/refunds`) → Razorpay Refund API → on Razorpay confirmation, `refunds.status='processed'`,

`payments.status='refunded' | 'partially_refunded'`, and the linked `provider_earnings.refund_amount`/`net_earning` are updated in the same transaction.

Failure handling: failed subscription renewal payments set `subscription_payments.status='failed'`; a retry policy (Section 17) attempts up to 3 retries over 3 days (`billingRetryJob.js`, cron), notifying the customer each attempt; after exhausting retries, subscription auto-transitions to `paused` with a `payment_failed` notification — never silently marked paid.

R. Wallet and Invoice Architecture

Wallet credits originate from: refunds routed to wallet (customer choice), promotional credits (admin `manual_admin_adjustment`), referral bonuses. Every mutation is `wallet_transactions` insert + `wallets.balance` update in one DB transaction.

Wallet balance can be applied at checkout as a partial payment method (deduct from `subtotal` before creating the Razorpay order for the remainder); if wallet fully covers the amount, no Razorpay order is created and `payments.status` can be marked `paid` via a `method='wallet'` payment row for consistent downstream accounting.

Invoices: generated synchronously right after payment success (Section Q step 5), numbered sequentially (`SH-INV-{year}-{seq}`), rendered to PDF via `pdfkit` and stored (path in `invoices.pdf_url`), downloadable via `GET /invoices/:id/pdf`.

S. Provider Accounting Architecture

Definitions (do not conflate — Section 5 critical rule):




Provider Earnings = Gross Sales – Platform Commission – Refunds
Net Profit = Provider Earnings – Provider Expenses

All four figures are computed by summing `provider_earnings` rows (for Gross Sales/Commission/Refunds/Earnings) and `provider_expenses` rows (for Expenses) **filtered by** `earning_date`/`expense_date` **within the requested period** — never hardcoded, never estimated.




sql

-- Example: period summary query shape

```
SELECT
SUM(gross_amount)           AS gross_sales,
SUM(commission_amount)      AS platform_commission,
SUM(refund_amount)          AS refunds,
SUM(net_earning)            AS provider_earnings
FROM provider_earnings
WHERE provider_id = :providerId AND earning_date BETWEEN :from AND :to;
```

```
SELECT COALESCE(SUM(amount),0) AS provider_expenses
FROM provider_expenses
WHERE provider_id = :providerId AND expense_date BETWEEN :from AND :to;
```

-- net_profit = provider_earnings - provider_expenses (computed in service layer, not SQL, for clarity/testability)

Settlement fields:

`Pending Settlement` = SUM(net_earning) WHERE settlement_id IS NULL AND earning_date <= today

`Available Settlement` = Pending Settlement minus any admin hold/threshold rules (v1: equal to Pending Settlement; hold logic is an admin

`platform_settings` toggle for future use)

`Paid Settlement` = SUM(total_earnings) WHERE settlements.status='paid'

Periods supported by `GET /provider/accounting/summary`: `today`, `yesterday`, `week`, `month`, `year`, `custom(from,to)` — all resolved server-side to date ranges in the provider's local (IST) timezone to avoid off-by-one-day errors around midnight.

T. Admin Commission and Settlement Architecture

Commission resolution hierarchy (most-specific wins), evaluated at the moment of payment capture:



1. Active commission_rules row WHERE scope='service' AND service_id = :serviceId
2. else Active commission_rules row WHERE scope='category' AND category_id = :categoryId
3. else Active commission_rules row WHERE scope='global'

"Active" = `[effective_from <= now() AND (effective_to IS NULL OR effective_to > now())]`. When admin updates a rule, the old row is closed (`[effective_to = now()]`) and a new row inserted — never UPDATE the `[commission_percent]` of an existing row in place, which is precisely what guarantees historical `[provider_earnings.commission_rate_applied]` snapshots remain correct even though the live rule changes (Section 47 requirement).

Settlement lifecycle: Provider `[POST /provider/settlements]` (requests settlement for a period) → server aggregates all unsettled `[provider_earnings]` in that period into a new `[provider_settlements]` row (`[status='requested']`) → Admin reviews `[GET/PUT /admin/settlements/:id]` → `[approved]` (locks the earnings rows via `[settlement_id]`) → funds transferred out-of-band → `[paid]` (with `[payout_reference]`) or `[rejected]` (unlocks earnings rows, `[settlement_id]` reset to NULL, reason recorded, audit-logged).

U. Notification and Support Architecture

In-app only for v1 (`[notifications]` table), architecture ready for email/SMS/push via a `[NotificationChannel]` interface (`[services/notificationService.js]` exposes `[notify(userId, type, payload)]`; internally fans out to `[InAppChannel]` now, `[EmailChannel]`/`[SmsChannel]`/`[PushChannel]` later — same call site, no controller changes needed later).

Notification `[type]` enum (matches Section 23): `[order_confirmed, subscription_created, subscription_paused, subscription_resumed, delivery_scheduled, delivery_out_for_delivery, delivery_completed, payment_successful, payment_failed, refund_processed, kyc_approved, kyc_rejected, settlement_updated, support_response]`.

Support: `[support_tickets]` + `[support_messages]`, threaded, `[Customer↔Admin]` and `[Provider↔Admin]` (not Customer↔Provider directly — keeps Admin as mediator per spec Section 24).

V. Multilingual Architecture

`[react-i18next]`, namespaced JSON files per bundle: `[locales/en/common.json]`, `[locales/en/customer.json]`, `[locales/en/provider.json]`, `[locales/en/admin.json]` (mirrored for `[hi]`, `[mr]`) — namespacing keeps translation files manageable rather than one giant file.

No hardcoded JSX strings — enforced via an ESLint rule (`[i18next/no-literal-string]`) in CI so this is a build-breaking check, not a convention people forget.

Language persisted in `[users.preferred_language]` (synced on change via `[PUT /customers/me]`) and mirrored to `[localStorage]` for pre-login state; `[i18next-browser-languagedetector]` picks it up on load.

Fallback: `[i18next]` `[fallbackLng: 'en']` — any missing `[hi]`/`[mr]` key renders the English string rather than the raw key.

Backend error messages also localized (`[Accept-Language]` header → localized `[message]` field in the error envelope) so validation/error toasts are translated too, not just static UI labels.

W. UI/UX and Page Architecture

Full page inventories per Sections 29–31 of the source spec are adopted as-is (Customer: Home/Services/Service Details/Search/Provider Details/About/Contact/FAQ/Privacy/Terms + auth pages + the full authenticated dashboard set; Provider:

Dashboard/Services/Orders/Deliveries/Calendar/Customers/Earnings/Analytics/Expenses/Settlements/Availability/Service Areas/KYC + support/profile; Admin: full console with Users/Providers/KYC/Categories/Subscriptions/Orders/Payments/Refunds/Settlements/Commissions/Coupons/Reviews/Support/Reports/Service Areas/Audit Logs/Settings).

Design direction: Customer = marketplace card grids with category color-coding (🗑️ 📦 💧 🪚), large touch targets, minimal cognitive load. Provider = dashboard shell (persistent left nav, KPI cards top-of-page, mobile-first per Section 53 — "today's income," "today's deliveries," and "add expense" reachable within 1-2 taps from the provider home screen). Admin = dense, information-rich console (data tables, filters, bulk actions) — visually the least "marketplace," most "enterprise back-office."

X. Security Architecture

Transport: HTTPS only in all non-local environments; HSTS via helmet.

`helmet()` defaults + explicit CSP; `cors()` locked to `CLIENT_URL` origin(s), credentials true only where cookies used (refresh token may be delivered as httpOnly secure cookie as an alternative to body-based storage — recommended over localStorage to reduce XSS exposure).

Input validation: Joi schemas on every mutating route, rejecting unknown fields (`stripUnknown` false → 400 on extras) to prevent mass-assignment.

SQL injection: fully mitigated by Sequelize parameterized queries; raw queries (if ever needed for reporting) must use bound replacements, never string concatenation.

XSS: React auto-escapes by default; any `dangerouslySetInnerHTML` usage (none expected) would require sanitization — explicitly disallowed unless reviewed.

CSRF: primarily mitigated by using `Authorization: Bearer` JWT (not ambient cookies) for state-changing requests; if refresh token is cookie-based, mark `SameSite=Strict` + CSRF token on refresh endpoint.

Rate limiting: global API limiter + stricter limiters on `/auth/*`.

File uploads: Multer with `fileFilter` (whitelist mime types per context — images for KYC/receipts/reviews, PDF for KYC), size limits (e.g. 5MB), and files stored outside the web-root execution path with randomized filenames.

Sensitive data: `RAZORPAY_KEY_SECRET`, `JWT_SECRET`, `JWT_REFRESH_SECRET`, `DB_PASSWORD` only in env vars, never logged, never returned in API responses.

`provider_bank_accounts.account_number_encrypted` via AES-256-GCM (app-layer key from env, distinct from JWT secret), only `account_number_last4` ever rendered to the UI. Password hashes never included in any serialized `User` response (explicit `attributes: { exclude: ['password_hash', 'refresh_token_hash'] }` on every relevant Sequelize query/scope).

Audit logging (Section 25/T/Q) covers every admin approval/rejection, commission change, refund, settlement decision, coupon create/modify, and user suspension.

Y. Testing Strategy

Unit (Jest): RBAC middleware behavior; subscription date/skip/vacation calculations; accounting formulas (`gross - commission - refund = earnings`, `earnings - expenses = profit`) against fixed fixtures; coupon discount math incl. min-order/max-discount caps; Razorpay signature verification (mocked HMAC); scheduler idempotency (`findOrCreate` doesn't double-insert on repeated run).

Integration (Jest + Supertest, test MySQL DB): register→login; full subscription-create→scheduler-run→delivery-generated chain; order-create→payment-verify→invoice-generated chain; provider accounting summary against seeded transactions; settlement request→approve→paid chain.

Frontend (Vitest + RTL): form validation states; protected-route redirects per role; language switch re-renders with translated strings; subscription pause/resume/skip UI flows (mocked API).

E2E (Playwright), matching Section 36 flows exactly: Customer register→search→subscribe→pay→track→review; Provider register→KYC→add service→receive order→complete delivery→add expense→view profit→request settlement; Admin login→approve provider→configure commission→monitor order→process settlement→view report.

CI gate: unit+integration required to pass before merge; E2E run nightly/pre-release against a seeded staging environment.

Z. Complete Development Roadmap

Development Order Rationale

The prompt's suggested order (Database → Backend foundation → Auth → RBAC → Core APIs → Subscription engine → Payment → Accounting → Customer UI → Provider UI → Admin UI → i18n → Testing → Deployment) is adopted **with one adjustment**: i18n scaffolding (namespaces, `t()` wiring, ESLint rule) is introduced starting in Phase 18 (first frontend phase) rather than bolted on at the end — retrofitting translations onto finished JSX after the fact is a common source of the exact "hardcoded strings in JSX" failure the spec explicitly forbids (Section 27). Translation *content* for hi/mr can still be finalized later, but the `t()`-key discipline must exist from the first component.

Gap Analysis (flagged per Section 59, nothing removed — only added)

`kyc_documents` **table** — the spec says "Upload KYC" and "Approve/reject KYC" but names no table; a provider needs multiple documents with independent review status. Added.

`service_plans` **table** — "Configure subscription plans" per service needs a place to hold multiple frequency/price tiers per service; folding this into `services` would prevent a service from having both Daily and Monthly pricing. Added.

`commission_rules` **table** — Section 47 requires a commission hierarchy with historical-accuracy guarantees; this requires a proper time-sliced rules table, not a single column on `categories`/`services`. Added.

`platform_settings` **key-value table** — "Manage platform settings" (Section 4/Admin) needs a persistence target; a generic settings table avoids a schema migration for every future toggle. Added.

Coupon funding ambiguity — the spec doesn't say whether a coupon discount is absorbed by the platform or the provider. **Resolution:** default to platform-funded (discount reduces customer payment, provider's `gross_amount` in `provider_earnings` is still the pre-discount service price, i.e., providers are made whole and the platform eats the coupon cost) unless a future `coupons.funded_by` extension is added. This must be confirmed with stakeholders before Phase 14; documented here as an explicit assumption, not silently decided.

Timezone handling — all `DATE`/`DATETIME` scheduling logic must run in **IST (Asia/Kolkata)** consistently (server, cron, and "today" boundary in accounting queries), regardless of underlying host TZ — enforced via `TZ=Asia/Kolkata` in deployment env and `moment-timezone`/`date-fns-tz` in date utils, to prevent the classic "delivery generated at 12:05 AM UTC lands on the wrong day for IST users" bug.

Duplicate subscription safety beyond delivery rows — the mandatory `UNIQUE(subscription_id, delivery_date)` protects deliveries; an equivalent idempotency guard is also needed on **billing** (`subscription_payments`) so a retried renewal job doesn't double-charge — implemented via `UNIQUE(subscription_id, billing_period_start)`. Added to the `subscription_payments` design intent (apply as a migration constraint).

Phase-by-Phase Implementation Plan

PHASE 0 — Requirements Finalization

Objective: Lock ambiguous business rules (coupon funding, settlement hold policy, tax applicability) with stakeholder sign-off; finalize this document as the frozen reference.

Prerequisites: none.

Deliverables: signed-off blueprint (this document), a short RFC resolving the coupon-funding assumption in Gap #5.

Acceptance criteria: no open business-rule ambiguity remains unassigned an explicit decision.

DoD: this document + RFC committed to repo `/docs`.

PHASE 1 — Project Setup & Architecture

Objective: Scaffold both repos (or monorepo) with the exact directory structures from Sections K/L.

Files/modules: `backend/` and `frontend/` skeletons, `package.json`, ESLint/Prettier configs, `.env.example` (Section AA), i18n lint rule installed (unused until Phase 18 but configured now).

Backend: `app.js` boots Express, `helmet`/`cors`/`rate-limit` wired with placeholder config; health-check route `GET /api/v1/health`.

Frontend: Vite React app boots, router shell, three placeholder layouts.

Testing: smoke test hitting `/health`.

Acceptance criteria: both apps run locally (`npm run dev`) against a local `.env`; CI pipeline scaffold (lint+test) green on empty test suite.

Common failure points: mismatched Node versions between devs — pin via `.nvmrc`/`engines`.

DoD: PR merged, CI green.

PHASE 2 — Database Design (Modeling, not migration yet)

Objective: Finalize ERD from Section I/J as Sequelize model files (no migrations run yet) — models + associations only, reviewed against this document table-by-table.

Database changes: none applied; `models/` written.

Acceptance criteria: `sequelize.sync({ validate: true })` (dev-only sanity check, never used in real envs) reports no association errors; every table in Section I has a matching model file.

DoD: model layer code-reviewed against Section I column-by-column.

PHASE 3 — MySQL + Sequelize Migrations

Objective: Write actual `migrations/*.js` files, one per table (or tightly related group), in FK-dependency order (roles→users→customers/providers→...→dependent tables last).

Database changes: all 31 tables created in a fresh MySQL 8 instance (cloud, SSL — Section 42) via `sequelize-cli db:migrate`.

Security: DB user for the app has least-privilege grants (no DROP/ALTER in production DB user, only via a separate migration-runner credential).

Testing: migration up/down round-trip tested on a throwaway schema.

Acceptance criteria: `db:migrate` succeeds cleanly on empty DB; `db:migrate:undo:all` cleanly reverses; all documented indexes/constraints (incl. mandatory `UNIQUE(subscription_id, delivery_date)`) present.

DoD: migration history reviewed; ERD diagram exported and stored in `/docs`.

PHASE 4 — Seed Data

Objective: Realistic Indian seed data per Section 51 — the four named providers (Amrut Pure Dairy, Annapurna Home Tiffin & Mess, HydroPure Mineral Water, ShineBright Professional Home Services), each with services/plans/service_areas/availability, several customers with addresses across those pincodes, sample subscriptions/orders/reviews/expenses.

Files: `seeders/*.js`, idempotent (safe to re-run against a clean DB).

Acceptance criteria: `db:seed:all` produces a demo-able platform (each category has ≥1 provider, ≥3 services, ≥2 active subscriptions).

DoD: seed reviewed for realism (no leftover generic "plumber/electrician" placeholder data, per explicit Section 51 exclusion).

PHASE 5 — Backend Foundation

Objective: Global middleware, error envelope, pagination utility, `utils/money.js`, logging.

Backend changes: `middlewares/errorHandler.js` producing the Section 35 envelope for all thrown errors; `utils/pagination.js` standard `?page&limit` handling.

Testing: unit tests for `money.js` (rounding/paise conversion) and the error envelope shape.

Acceptance criteria: any thrown `AppError(code,message)` anywhere in the stack surfaces as the correct JSON envelope + HTTP status.

DoD: reviewed, documented in `/docs/error-codes.md`.

PHASE 6 — Authentication + RBAC

Objective: Implement Section N fully.

APIs: `/auth/register|login|refresh|logout|forgot-password|reset-password`.

Security: bcrypt cost 12, rate limiting on auth routes, refresh rotation.

Testing: unit (token issuance/verification, RBAC middleware), integration (full register→login→refresh→logout).

Acceptance criteria: an unauthenticated request to any protected route returns 401; a customer-token request to a provider-only route returns 403; refresh rotation invalidates the prior refresh token.

DoD: security review of token TTLs and storage.

PHASE 7 — Customer APIs (core, non-financial)

Objective: profile, addresses, service browsing/search+pincode filter, provider public profile.

Database: none new (uses Phase 3 schema).

Testing: integration tests for address CRUD ownership, pincode-filtered service search correctness.

Acceptance criteria: searching `?category=milk&pincode=X` returns only providers with a matching `service_areas` row.

DoD: reviewed against Section E.1 journey start.

PHASE 8 — Provider APIs (core, non-financial)

Objective: provider profile, KYC upload, services/service_plans CRUD, service_areas, availability — all gated by `kyc_status='approved'` for write access to services/plans (KYC upload itself allowed pre-approval).

Testing: integration test that a `pending` provider gets 403 on service creation; `approved` provider succeeds.

Acceptance criteria: matches Section E.3 journey through "Add Services."

DoD: reviewed.

PHASE 9 — Admin APIs (core, non-financial)

Objective: user/customer/provider listing, KYC approve/reject (writes `audit_logs`), category/service moderation.

Testing: integration test KYC approval flips `providers.kyc_status` and creates an `audit_logs` row with correct old/new values.

Acceptance criteria: matches Section E.5 journey through "Approve/Reject."

DoD: reviewed.

PHASE 10 — Service Management (full CRUD + attributes_json validation per category)

Objective: category-specific attribute validation (milk_type/meal_type/water_size/cleaning_type enums enforced via Joi per `category.slug`).

Testing: unit tests per category's attribute schema.

Acceptance criteria: invalid attribute combos (e.g., a Milk service with a `cleaning_type` field) rejected with 422.

DoD: reviewed.

PHASE 11 — Subscription Engine (CRUD + state machine, no scheduler yet)

Objective: `customer_subscriptions` CRUD, pause/resume/cancel/skip/vacation endpoints implementing the Section O state machine, with service-area validation enforced at creation.

Testing: unit tests for every state transition (including illegal transitions rejected, e.g. resuming a cancelled subscription).

Acceptance criteria: full Section O diagram covered by tests.

DoD: reviewed.

PHASE 12 — Delivery Scheduler

Objective: implement Section P exactly — `jobs/subscriptionDeliveryJob.js`, node-cron wiring, manual trigger endpoint.

Testing: unit test idempotency (run twice same day → one delivery row); integration test with seeded active/paused/vacation/skipped subscriptions producing the correct delivery set.

Acceptance criteria: zero duplicate `(subscription_id, delivery_date)` rows possible even under concurrent trigger.

DoD: reviewed, TZ handling (Gap #6) explicitly tested.

PHASE 13 — Order Management (one-time: Cleaning + emergency Water)

Objective: `orders`/`order_items` CRUD, availability-slot validation for Cleaning, server-side total computation.

Testing: integration test that client-submitted totals are ignored/recomputed server-side.

Acceptance criteria: matches Section E.2 journey through order creation (pre-payment).

DoD: reviewed.

PHASE 14 — Razorpay Payments

Objective: full Section Q implementation — create-order, verify, webhook, refund initiation.

Security: signature verification unit-tested against known Razorpay test vectors; webhook signature verification separately tested.

Acceptance criteria: an order/subscription is only ever marked paid/confirmed after successful signature verification (client-only success is provably insufficient — test explicitly asserts a forged/omitted signature does not confirm payment).

DoD: reviewed against Section 46 (financial safety) checklist.

PHASE 15 — Wallet + Invoices

Objective: wallet ledger + checkout integration, PDF invoice generation.

Acceptance criteria: wallet balance always equals SUM of its `wallet_transactions`; invoice PDF downloadable post-payment.

DoD: reviewed.

PHASE 16 — Provider Accounting

Objective: `provider_earnings` insertion wired into payment-success path (Phase 14) and delivery-completion path (Phase 12); `GET /provider/accounting/summary` implementing Section S formulas exactly.

Testing: the critical Section 5 example (Gross ₹10,000 / Commission ₹1,000 / Refunds ₹500 / Earnings ₹8,500 / Expenses ₹2,000 / Net Profit ₹6,500) encoded as a fixture-based unit test.

Acceptance criteria: Earnings and Profit are never conflated anywhere in API responses or UI copy.

DoD: reviewed by a second engineer specifically for the earnings/profit distinction.

PHASE 17 — Provider Settlements

Objective: Section T settlement lifecycle, admin approve/reject/paid endpoints, commission-rule time-slicing (Gap #3) implemented and tested for historical-accuracy (changing a rule must not alter past `provider_earnings.commission_rate_applied`).

Acceptance criteria: matches Section E.4 journey conclusion + Section E.5 "Process Settlement."

DoD: reviewed, audit_logs coverage confirmed.

PHASE 18 — Customer Frontend

Objective: build all Customer pages (Section 29 inventory), wire to APIs from Phases 7/11/13/14/15, i18n key discipline established from day one (ESLint rule active).

UI: marketplace-style, mobile-first, loading/error/empty states + skeletons (Section 35).

Testing: RTL tests for subscription create/pause/skip flows (mocked API); Vitest for form validation.

Acceptance criteria: full Section E.1/E.2 customer journeys click-through-able end to end against the real backend (staging).

DoD: reviewed for accessibility basics (Section 54) and for zero raw string literals in JSX (lint-enforced).

PHASE 19 — Provider Frontend

Objective: full Provider dashboard (Section 30 inventory) including the mobile-priority home screen (Section 53: today's income/deliveries reachable in 1-2 taps).

Testing: RTL tests for expense CRUD, accounting summary rendering, delivery status update flow.

Acceptance criteria: full Section E.3/E.4 journeys click-through-able.

DoD: reviewed on a narrow viewport for the mobile-priority requirement specifically.

PHASE 20 — Admin Frontend

Objective: full Admin console (Section 31 inventory), dense data tables, KYC review UI, commission/settlement management UI, audit log viewer.

Testing: RTL tests for role-protection (non-admin JWT can't render admin routes even if navigated directly), KYC approval flow.

Acceptance criteria: full Section E.5 journey click-through-able.

DoD: reviewed.

PHASE 21 — Multilingual Content Completion

Objective: fill in `hi.json`/`mr.json` content across all namespaces (structure already existed since Phase 18 — this phase is translation content + native-speaker review, not architecture).

Testing: automated check that every key present in `en/*.json` exists (or intentionally falls back) in `hi/*.json` and `mr/*.json`.

Acceptance criteria: manual language-switch QA pass across all three roles finds zero raw English leakage outside intentional proper nouns (brand name, ₹ symbol).

DoD: native speaker sign-off for Hindi and Marathi.

PHASE 22 — Notifications + Support

Objective: Section U — full notification type coverage wired at every trigger point named in Section 23; support ticket/message UI + APIs.

Testing: integration test that each of the 13 notification types fires at its correct trigger (e.g. `payment_failed` fires from the billing-retry job).

Acceptance criteria: notification bell shows unread count; support thread round-trips Customer↔Admin and Provider↔Admin.

DoD: reviewed.

PHASE 23 — Reports + Exports

Objective: Admin `/admin/reports/*` (revenue, growth, category performance, cancellation/completion rates) and Provider analytics charts (Section 7),

CSV export for expenses (Section 6) and reports.

Testing: report totals cross-checked against raw ledger sums in integration tests.

Acceptance criteria: every analytics number traces to a real MySQL aggregation query, none hardcoded (explicit code review checklist item per Section 7/46).

DoD: reviewed.

PHASE 24 — Security Audit

Objective: full pass against Section X checklist; dependency vulnerability scan; verify no secrets in repo/logs; confirm bank-account encryption at rest and masking in every response path.

Testing: automated secret-scanning in CI; manual pen-test-style pass on auth/payment/refund endpoints (IDOR checks — can customer A fetch customer B's order by guessing ID?).

Acceptance criteria: zero critical/high findings open.

DoD: signed-off security checklist committed to `/docs`.

PHASE 25 — Testing (Full Suite Hardening)

Objective: fill any coverage gaps from Section Y across all prior phases; wire Playwright E2E for the three full journeys (Section 36).

Acceptance criteria: CI coverage thresholds met (recommend ≥80% on `services/` financial/subscription logic specifically, not just aggregate).

DoD: CI green including nightly E2E.

PHASE 26 — Performance Optimization

Objective: verify indexes from Section I are actually used (`EXPLAIN` on hot queries — accounting summary, service search, delivery scheduler scan); add API-level pagination everywhere lists are returned; frontend code-splitting per role bundle; image optimization for provider/service images.

Acceptance criteria: accounting summary and service-search endpoints respond within an agreed SLA (e.g. <300ms p95) against seeded realistic-scale data (10k+ rows per hot table).

DoD: perf test report committed.

PHASE 27 — Deployment

Objective: execute Section AC (Deployment Strategy) for staging then production.

Acceptance criteria: staging environment fully functional with production-equivalent config (SSL DB, real Razorpay test-mode keys, real domain with HTTPS).

DoD: staging sign-off by product owner.

PHASE 28 — Production Readiness

Objective: cutover checklist (Section AD Production Checklist), Razorpay **live** credentials swap, monitoring/alerting live, backup schedule verified with a real restore drill.

Acceptance criteria: Section 56 Final Acceptance Criteria (all Customer/Provider/Admin/System items) verified in production or production-mirrored environment.

DoD: go-live sign-off.

AA. Development Environment Setup

Install Node LTS (pin via `.nvmrc`), MySQL client tools (for `mysql`/`mysqldump`), even though DB is cloud-hosted).

Clone repo, `cp backend/.env.example backend/.env` and `cp frontend/.env.example frontend/.env`; fill in real values (never commit `.env`).

```
cd backend && npm install && npm run db:migrate && npm run db:seed
```

```
cd frontend && npm install
```

`npm run dev` in both — backend on e.g. `:5000`, frontend on `:5173` (Vite default), `VITE_API_BASE_URL=http://localhost:5000/api/v1`.

Recommended editor tooling: ESLint + Prettier extensions, `.editorconfig` committed.

AB. MySQL Setup

Local dev option: developers may run a local MySQL 8 instance for offline work, but CI/staging/prod always target the cloud-hosted MySQL (Section 42) — connection config must not assume `localhost` (Section 42 explicit requirement).

Cloud MySQL connection (Sequelize `config/db.js`):



```
js
```

```

module.exports = {
  dialect: 'mysql',
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  username: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  dialectOptions: {
    ssl: { require: true, rejectUnauthorized: true } // SSL required per Section 42
  },
  pool: { max: 10, min: 0, acquire: 30000, idle: 10000 },
  retry: { max: 3 } // connection retry on transient network errors
};

```

Connection pooling: tune `pool.max` per environment (higher in production behind a load-balanced backend, capped to stay under the managed MySQL provider's max-connections limit across all app instances).

Retries: wrap initial connection bootstrap in a retry-with-backoff loop (e.g. 5 attempts, exponential) so a transient cloud-DB blip at container start doesn't crash-loop the app.

Backup/recovery: rely on the managed MySQL provider's automated daily snapshots at minimum; additionally schedule a `mysqldump` export to object storage weekly as a provider-independent recovery path; document and periodically test the restore procedure (Phase 28 requires an actual restore drill, not just a backup that's never been verified restorable).

AC. Sequelize Migration Strategy

Naming: `sequelize-cli migration:generate --name <ordinal>-create-<table>` → files like `20260101010101-create-users.js`, timestamp prefix guarantees FK-safe ordering.

Ordering: roles → users → customers/providers → addresses/kyc_documents/provider_bank_accounts/provider_availability → categories → services → service_plans/service_areas → customer_subscriptions → subscription_deliveries/skipped_deliveries → orders → order_items → payments → subscription_payments (needs payments FK, so after payments) → provider_earnings (needs payments) → refunds → invoices → coupons → coupon_usage → reviews → notifications → support_tickets → support_messages → audit_logs → commission_rules → platform_settings → provider_settlements (needs provider_earnings) → wallets/wallet_transactions.

Rollback: every migration implements both `up` and `down`; `down` must fully reverse `up` (drop tables/columns/constraints in reverse dependency order). Never edit a migration that has run in any shared environment — write a new migration to alter it instead.

Environments: separate `config/config.json`/env-driven configs for `development`, `test`, `production` — test environment points at an isolated MySQL schema (created/torn down per CI run or per test suite via `db:migrate`/`db:migrate:undo:all`).

Verification: CI step runs `db:migrate` against a fresh schema on every PR to catch migration errors before merge, independent of the seed/test steps.

AD. Seed Data Strategy

Idempotent seeders (checked via `findOrCreate` on natural keys like email/business_name) so `db:seed:all` is safe to re-run.

Seed order mirrors migration order; seeders reference each other's inserted IDs directly within the same seeder run (no hardcoded IDs assumed).

Distinct seed sets: `development` (rich, Section 51 realistic Indian data — the four named providers, multiple customers, active subscriptions/orders/reviews/expenses spanning several months for meaningful analytics charts) vs `test` (minimal, deterministic fixtures for fast integration tests, separate from the demo dataset).

AE. Environment Variables (`.env.example`)



Server

PORT=5000

CLIENT_URL=http://localhost:5173

NODE_ENV=development

TZ=Asia/Kolkata

Database (MySQL, cloud, SSL required)

DB_HOST=

DB_PORT=3306

DB_NAME=servicehub

DB_USER=

DB_PASSWORD=

DB_DIALECT=mysql

Auth

JWT_SECRET=

JWT_REFRESH_SECRET=

JWT_ACCESS_EXPIRY=15m

JWT_REFRESH_EXPIRY=7d

Razorpay

RAZORPAY_KEY_ID=

RAZORPAY_KEY_SECRET=

RAZORPAY_WEBHOOK_SECRET=

App-layer encryption (bank account numbers)

BANK_ACCOUNT_ENCRYPTION_KEY=

File uploads

```
UPLOAD_DIR=./uploads
MAX_UPLOAD_SIZE_MB=5
```

(Frontend `.env.example`: `VITE_API_BASE_URL=`, `VITE_RAZORPAY_KEY_ID=` — public key only, never the secret.)

No real secrets committed; `.env` in `.gitignore`.

AF. Git/GitHub Strategy

Trunk-based with short-lived feature branches: `feature/<phase-number>-<short-desc>` (e.g. `feature/12-delivery-scheduler`), merged via PR into `main` (or `develop` if a staging-gate branch is desired).

Conventional commits (`feat:`, `fix:`, `chore:`, `test:`) to enable changelog generation.

Branch protection on `main`: required CI (lint+unit+integration), required review, no direct pushes.

PR template requiring: which phase/section this addresses, DB migration included y/n, tests added y/n.

Tag releases per deployed version (`v0.1.0` at end of staging Phase 27, `v1.0.0` at production go-live Phase 28).

AG. Deployment Strategy

Development: local machines + shared dev MySQL (optional) or local MySQL; hot-reload dev servers.

Testing/Staging:

Backend: containerized (Docker) deploy to a staging host/PaaS; env vars injected via the platform's secret manager, not baked into the image.

Frontend: `vite build` static output deployed to a static host/CDN; `VITE_API_BASE_URL` points at staging backend.

MySQL: staging schema on the same cloud MySQL provider (separate database/instance from production), SSL enforced.

Razorpay: **test-mode** keys only.

HTTPS: staging subdomain with a valid TLS cert (e.g. via the hosting platform or Let's Encrypt/reverse proxy).

CORS: `CLIENT_URL` restricted to the staging frontend origin.

File storage: staging uses the same Multer-disk-with-abstraction-layer approach on a persistent volume (not ephemeral container storage, or uploads vanish on redeploy).

Logs/monitoring: centralized log aggregation (even a simple hosted log service) + uptime check on `/health`.

Backups: staging DB snapshotted daily (lower retention than prod is acceptable).

Migrations: run explicitly as a deploy step (`db:migrate`) **before** the new app version starts serving traffic, never on app boot implicitly (avoids race conditions with multiple instances starting concurrently).

Rollback: keep the previous container image tagged and deployable; migrations should prefer additive/backward-compatible changes per release so a code rollback doesn't require an immediate migration rollback too.

Production:

All of the above, plus: **live** Razorpay credentials (swapped only after full staging sign-off), custom domain with HTTPS enforced end-to-end, stricter rate limits, real monitoring/alerting (error-rate, latency, DB connection pool saturation, failed-payment-rate alerts), automated daily backups with tested restore, and a documented on-call/incident process for payment-related failures specifically (given Section 46's financial-safety emphasis).

AH. Production Checklist

- ☐ All 28 phases' acceptance criteria met and signed off.
- ☐ Razorpay live keys configured; test-mode keys removed from prod env.
- ☐ HTTPS enforced on all origins; HSTS enabled.
- ☐ DB backups automated + restore drill completed successfully within the last 30 days.
- ☐ Security audit (Phase 24) findings closed.
- ☐ Rate limiting tuned for expected production traffic (not dev defaults).
- ☐ All secrets rotated from any values used during development/staging.
- ☐ Monitoring/alerting live and tested (trigger a synthetic failure, confirm alert fires).
- ☐ i18n content (hi/mr) reviewed by native speakers, zero leaked raw keys or English strings in production build.
- ☐ Legal pages (Privacy, Terms) reviewed by a human, not placeholder text.
- ☐ Bank account encryption verified end-to-end (insert → encrypted at rest confirmed via direct DB inspection → masked correctly in every API response and UI screen).
- ☐ Load/perf test at expected launch scale passed (Phase 26 SLAs met).
- ☐ Support ticket routing confirmed reaching real admin staff.
- ☐ Rollback procedure documented and rehearsed at least once in staging.

AI. Risk Register

Risk	Likelihood	Impact	Mitigation
Duplicate subscription deliveries from concurrent scheduler triggers	Medium	High (double-charges customer trust, provider confusion)	<code>UNIQUE(subscription_id, delivery_date)</code> constraint (mandatory, Section 12) + <code>findOrCreate</code>
Client-reported payment success trusted, no real charge	Low if followed	Critical (revenue loss/fraud)	Server-side signature verification + webhook as source-of-truth backstop (Section Q)
Commission rate change retroactively alters historical provider earnings	Medium	High (provider trust/legal/accounting disputes)	Snapshot <code>commission_rate_applied</code> on <code>provider_earnings</code> at transaction time (Section T)
Floating-point rounding errors in money math	Medium	Medium (accounting mismatches, audit failures)	<code>DECIMAL(12,2)</code> columns + integer-paise arithmetic in <code>utils/money.js</code> (Section 46)
Provider serves an area they've since removed from <code>service_areas</code> , but active subscriptions remain	Medium	Medium (deliveries generated for uncovered areas)	Scheduler checks area coverage every run, flags + notifies admin instead of silently generating (Section P)
Timezone bugs around midnight IST	Medium	Medium (wrong-day deliveries/billing)	<code>TZ=Asia/Kolkata</code> enforced everywhere, explicitly tested (Gap #6)
Coupon abuse (same user, multiple accounts, or exceeding per-user limit)	Medium	Low-Medium	<code>coupon_usage</code> unique tracking + per-user limit enforcement (Section 21)
Bank account data exposure	Low	Critical (regulatory/trust)	AES-256 encryption at rest, last-4-only in any response, access audited
Scheduler job failure for one subscription blocks the entire daily run	Low if implemented correctly	High (no deliveries generated platform-wide)	Per-subscription try/catch isolation, failures logged not fatal (Section P)

Risk	Likelihood	Impact	Mitigation
Settlement double-payment (provider requests overlapping periods)	Low	Medium	<code>provider_earnings.settlement_id</code> lock on approval prevents re-inclusion in a second settlement
i18n regression (hardcoded string slips into JSX)	Medium	Low-Medium (poor UX for hi/mr users, but not functionally breaking)	CI-enforced ESLint rule (Section V)

AJ. Common Failure Points

Forgetting to recompute totals server-side and trusting a client-submitted `amount`/`total` (Phase 13/14 — explicitly tested against).

Running the delivery scheduler without the `UNIQUE` constraint in place first (must ship in the same migration set, Phase 3, before Phase 12 is ever exercised against real data).

Treating `wallets.balance` as writable directly instead of always deriving it from a `wallet_transactions` insert within the same DB transaction.

Mixing "Provider Earnings" and "Net Profit" terminology in UI copy or API field names (explicit code-review checklist item, Phase 16/19).

Storing bank account numbers in plaintext "temporarily" during development and forgetting to encrypt before any real data enters the table.

Applying `commission_rules` changes via UPDATE instead of closing the old rule (`effective_to`) and inserting a new one — silently corrupts historical accuracy.

Skipping the `TZ=Asia/Kolkata` setting on a staging/production host, causing deliveries to generate a day early/late relative to what IST users expect.

Building the Admin frontend to look identical to Provider/Customer (violates Section 28/52 explicit differentiation requirement) — a design-review checkpoint should catch this at Phase 20.

AK. Troubleshooting Plan

Payments show "paid" on frontend but no `provider_earnings` row exists → check webhook delivery/logs first (client-only `/verify` call may have failed silently on a flaky connection); replay the webhook event manually if the platform's Razorpay dashboard allows it; never manually flip `payments.status` without corresponding signature evidence.

Missing deliveries for a date → check `jobs` run log for that date; check if the subscription was `paused`/`vacation`/`skipped` (expected) vs. an actual job failure (check per-subscription try/catch error log); re-run `POST /subscriptions/run-scheduler` manually with a `targetDate` param for backfill if a genuine job failure occurred (idempotent, safe to re-run).

Accounting numbers "look wrong" to a provider → reconcile by re-running the raw SQL aggregation (Section S) directly against `provider_earnings`/`provider_expenses` for the disputed period and compare to the API response; check for a recent commission-rule change and confirm it did *not* alter the `commission_rate_applied` snapshot on pre-existing rows (it must not have).

Migration fails partway in production → never `--force` past a failed migration; roll back the specific migration's `down`, fix, redeploy; this is why migrations should be small and reviewed (Phase 3/Section AC).

Language shows raw translation keys → check the specific namespace file for the missing key in that locale; confirm `fallbackLng` config is intact; this should have been caught by CI's key-parity check (Section AD) — treat a production occurrence as a CI gap to fix.

Final Definition of Done (Project-Level)

ServiceHub is DONE when, in a production-equivalent environment:

Every item in Section 56 (Final Acceptance Criteria: Customer/Provider/Admin/System) has been manually verified end-to-end, not just unit-tested in isolation.

All 28 phases' individual acceptance criteria are satisfied and their code has passed review against this document's specific formulas, constraints, and state machines (not just "looks reasonable").

The Production Checklist (Section AH) is fully checked.

The Risk Register (Section AI) mitigations are implemented and verifiable in code/config, not just documented intentions.

Documentation (README, installation, DB/migration/seed guides, API docs, Razorpay integration guide, deployment guide, testing guide, troubleshooting guide, business rules doc) exists and is accurate against the current codebase.

End of ServiceHub Master Roadmap & Preparation Plan.